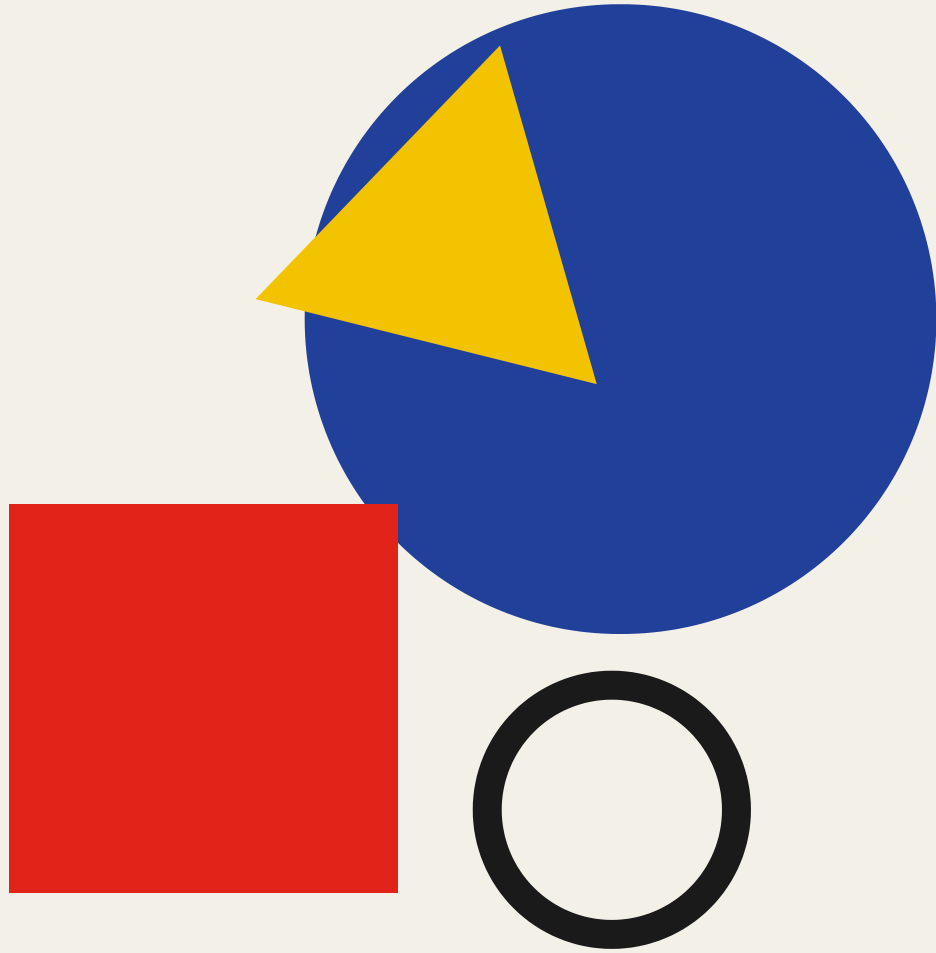


A FIELD GUIDE

get off the platform.

self-host your app and own your stack.

get off the platform: self-host your app and own your stack

Who this is for: You built an app on Vercel, Replit, Railway, or Render. It works. Then you hit the wall — $20/month became $200, your database is locked in, you can't run a background worker without a paid tier, and you have zero visibility into what's actually happening. This guide gets you off the platform and onto your own VPS, with every command you need to run, for $5-10/month. No DevOps background required.

---

1. when to leave (and when to stay)

You're ready for a VPS if you check 3+ of these: - [ ] Your hosting bill exceeds $25/month - [ ] You need a cron job, background worker, or WebSocket that the platform locks behind a paywall - [ ] Your app has been rate-limited by the platform itself - [ ] You want to use a database that isn't the platform's default - [ ] You need to comply with data residency requirements (GDPR, HIPAA-lite) - [ ] You're tired of waking up to "Your function has been suspended" emails

Stay on the platform if you're pre-launch, have no users, and your monthly bill is under $10. The platform handles things you don't want to think about yet. Come back to this guide when it hurts.

---

2. picking a vps: $5/month gets you far

PROVIDER	CHEAPEST TIER	RAM	CPU	BEST FOR
Contabo Cloud VPS 10	$5.28/mo	8 GB	4 vCPU	Most power per dollar
Hetzner	€4.50/mo	4 GB	2 vCPU	Best value in the EU
DigitalOcean	$6/mo	1 GB	1 vCPU	Best docs + ecosystem
Linode	$5/mo	1 GB	1 vCPU	Solid, Akamai-backed
Vultr	$6/mo	1 GB	1 vCPU	Good global regions

The value pick is clear: **Contabo Cloud VPS 10** gives you 4 vCPU and 8 GB RAM for about $6/month ($5.28 on a yearly term, $6.60 month to month). That is 4 to 8 times the RAM and CPU you get from DigitalOcean, Linode, or Vultr at the same price, and that headroom is exactly what you need the moment you add a database, a background worker, or anything AI-adjacent. In the EU,

Hetzner is the other strong choice. All of them run Ubuntu 22.04 LTS, so use that. Every command in this guide assumes Ubuntu 22.04.

provisioning in 5 minutes

```
# After creating your droplet/VPS (DigitalOcean example):
ssh root@your-server-ip

# First things first — update and create your user
apt update && apt upgrade -y
adduser deploy
usermod -aG sudo deploy
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy

# Log out and back in as deploy
exit
ssh deploy@your-server-ip
```

---

3. domain + dns — point your name at your server

```
# In your DNS provider (Namecheap, Cloudflare, Porkbun):
# Add an A record:
#   Type: A
#   Name: @
#   Value: <your-server-ip>
#   TTL: 300

# Add a wildcard for subdomains:
#   Type: A
#   Name: *
#   Value: <your-server-ip>
```

Wait 5-60 minutes for DNS propagation, then verify:

```
dig +short yourdomain.com
# Should return your server IP
```

---

4. docker — ship your app the same way everywhere

Docker is the bridge between “it works on my machine” and “it works on the server.” AI assistants can generate a Dockerfile for your app.

```
# Install Docker on Ubuntu 22.04
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee
/etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo usermod -aG docker $USER
# Log out and back in for group to take effect
```

a production-ready docker compose file

```
# docker-compose.yml
version: '3.8'
services:
  app:
    build: .
    restart: unless-stopped
    ports:
      - "3000:3000"
    env_file:
      - .env
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s
      timeout: 5s
      retries: 3

  db:
    image: postgres:16-alpine
    restart: unless-stopped
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  pgdata:
```

```
# Deploy
docker compose up -d --build
docker compose logs -f   # watch the logs
```

sqlite — the zero-ops database

For smaller apps or single-server deployments, skip Postgres entirely:

```
services:
  app:
    volumes:
      - ./data:/app/data   # SQLite file lives here — BACK THIS UP
```

SQLite handles way more than people think. If you have < 100K records and write-heavy isn't your bottleneck, it's simpler, faster to deploy, and requires zero container orchestration.

---

5. reverse proxy + tls — your app on the public internet

Caddy is the simplest reverse proxy on earth. It auto-provisions TLS certificates, auto-renews them, and redirects HTTP to HTTPS — all in 4 lines of config.

```
# Install Caddy
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo tee
/etc/apt/keyrings/caddy-stable.asc
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee
/etc/apt/sources.list.d/caddy-stable.list
sudo apt update
sudo apt install -y caddy
```

caddyfile — the whole config

```
# /etc/caddy/Caddyfile
yourapp.com {
    reverse_proxy localhost:3000
}

api.yourapp.com {
    reverse_proxy localhost:3001
}
```

```
sudo systemctl reload caddy
```

That's it. Your app is now at `https://yourapp.com` with valid TLS. Certificates auto-renew. No cron job, no certbot, no Let's Encrypt CLI. Caddy handles everything.

if you prefer nginx + certbot

```
sudo apt install -y nginx certbot python3-certbot-nginx
sudo certbot --nginx -d yourapp.com -d www.yourapp.com
# Certbot auto-renews via systemd timer — check it:
sudo systemctl status certbot.timer
```

Nginx config for proxying:

```
# /etc/nginx/sites-available/yourapp
server {
    listen 443 ssl;
    server_name yourapp.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

---

6. backups — the thing you'll regret skipping

database backups

```
# PostgreSQL — nightly dump
# Save as /etc/cron.d/db-backup
0 3 * * * deploy pg_dump -U appuser appdb | gzip > /home/deploy/backups/db-$(date
+\%Y\%m\%d).sql.gz

# SQLite — even simpler
0 3 * * * deploy cp /home/deploy/app/data/production.db /home/deploy/backups/db-$(date
+\%Y\%m\%d).db
```

off-server backups to s3

```
# Install awscli
sudo apt install -y awscli
aws configure  # enter your IAM credentials (with write-only S3 access)
```

```
# Cron job to sync backups to S3
0 4 * * * deploy aws s3 sync /home/deploy/backups/ s3://your-bucket/backups/ --delete
```

Or use `rclone` for Backblaze B2 ($5/TB, way cheaper than S3):

```
sudo apt install -y rclone
rclone config  # interactive setup for B2
# Cron:
0 4 * * * deploy rclone sync /home/deploy/backups/ b2:your-bucket/backups/
```

test your backups

A backup you haven't restored is not a backup. Once a month:

```
# Restore the latest backup to a test database
gunzip -c /home/deploy/backups/db-20260601.sql.gz | psql -U appuser test_restore_db
# Run your app against test_restore_db and verify it works
```

---

7. basic hardening — keep the script kiddies out

firewall

```
sudo ufw allow 22/tcp     # SSH
sudo ufw allow 80/tcp     # HTTP
sudo ufw allow 443/tcp    # HTTPS
sudo ufw enable
sudo ufw status verbose
```

ssh hardening

Edit `/etc/ssh/sshd_config`:

```
PermitRootLogin no          # never allow root SSH
PasswordAuthentication no   # keys only
MaxAuthTries 3              # limit brute force attempts
```

```
sudo systemctl reload sshd
```

fail2ban — auto-block brute forcers

```
sudo apt install -y fail2ban
sudo systemctl enable fail2ban --now
```

```
# Default config blocks IPs after 5 failed SSH attempts for 10 minutes
# Check who's been banned:
sudo fail2ban-client status sshd
```

automatic security updates

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades
# Choose "Yes" to enable automatic security updates
```

---

8. migration walkthrough — vercel → vps

Here's how to move a typical Next.js app from Vercel to your own server.

step 1: dockerize the app

```
# Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public
COPY --from=builder /app/package*.json ./
RUN npm ci --production
EXPOSE 3000
CMD ["npm", "start"]
```

step 2: move environment variables

Export from Vercel dashboard → copy to `.env` on your server:

```
# .env on your server
DATABASE_URL=postgresql://...
NEXTAUTH_SECRET=...
NEXT_PUBLIC_APP_URL=https://yourapp.com
```

step 3: set up the database

If you were using Vercel Postgres, dump and restore:

```
# Dump from Vercel (run locally)
pg_dump $VERCEL_PG_URL > vercel_dump.sql

# Copy to server and restore
scp vercel_dump.sql deploy@your-server:/tmp/
ssh deploy@your-server "psql -U appuser appdb < /tmp/vercel_dump.sql"
```

step 4: deploy

```
docker compose up -d --build
docker compose logs -f   # watch for errors
```

step 5: switch dns

Update your A record to point at the VPS IP. Keep the Vercel deployment running for 24 hours as a fallback (reduce TTL to 60 seconds before switching).

---

9. ci/cd — deploy on push without a platform

You don't need Vercel's Git integration to auto-deploy. GitHub Actions + a VPS gives you the same push-to-deploy workflow for free.

github actions → vps deploy

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to VPS
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.VPS_HOST }}
          username: deploy
          key: ${{ secrets.VPS_SSH_KEY }}
          script: |
            cd /home/deploy/app
            git pull origin main
            docker compose up -d --build
            docker image prune -f
```

Add `VPS_HOST`, `VPS_SSH_KEY` to your GitHub repo's Settings → Secrets. Now every push to main triggers a deploy. Same experience as Vercel, on your own hardware.

zero-downtime with docker

```
# In your deploy script:
docker compose up -d --build --scale app=2 --no-recreate
# Wait for the new container to be healthy, then:
docker compose up -d --scale app=1
```

---

10. monitoring — know when things break

```
# Basic health check — add to cron every 5 minutes
*/5 * * * * deploy curl -sf https://yourapp.com/health || echo "DOWN" | mail -s "App
down" you@email.com
```

For real monitoring, add Uptime Kuma (free, self-hosted):

```
# docker-compose.yml addition
  uptime-kuma:
    image: louislam/uptime-kuma:latest
    restart: unless-stopped
    ports:
      - "3001:3001"
    volumes:
      - kuma-data:/app/data
```

Browse to `http://your-server:3001`, set up a monitor for `https://yourapp.com`, and configure Telegram/Discord/email alerts.

resource monitoring

```
# Real-time: htop (interactive)
sudo apt install -y htop

# Disk usage alert — cron every hour
0 * * * * deploy [ $(df / | tail -1 | awk '{print $5}' | tr -d '%') -gt 80 ] && echo
"Disk > 80%" | mail -s "Disk alert" you@email.com

# Docker cleanup — weekly, prevent disk fill
@weekly deploy docker system prune -af --volumes
```

---

11. what this costs (real numbers)

SERVICE	PLATFORM COST	VPS COST
Hosting (Next.js)	Vercel Pro $20/mo	Hetzner $5/mo
Database	Vercel Postgres $20/mo	Self-hosted Postgres $0
Background jobs	Inngest $20/mo	Cron on VPS $0
File storage	Vercel Blob $20/mo	S3 $3/mo or local disk $0
Monitoring	Vercel Analytics $20/mo	Uptime Kuma $0
Total	**$100/mo**	**$8/mo**

The break-even on learning Docker + Caddy + UFW is roughly one weekend. After that, you're saving $90/month forever. For that price you could run three redundant servers and still come out ahead.

---

12. the migration checklist

```
☐ VPS provisioned and accessible via SSH
☐ Non-root user created, SSH keys working
☐ Docker and docker compose installed
☐ App Dockerized and running locally with `docker compose up`
☐ Environment variables in .env (NOT committed)
☐ Database migrated (dump → restore, or fresh setup)
☐ Caddy or Nginx proxying to your app on :3000
☐ TLS working — https://yourapp.com shows the green lock
☐ DNS updated and propagated
☐ Firewall enabled (22, 80, 443 only)
☐ SSH hardened (no root login, key-only)
☐ Fail2ban running
☐ Daily database backups configured and tested
☐ Off-server backup (S3/B2) configured
☐ Health check monitoring active
☐ Vercel deployment still running (keep for 24h fallback)
☐ After 24h of stable VPS: cancel Vercel project
```

You don't need a DevOps team to own your stack. You need one VPS, Docker, Caddy, and this guide. The $90/month you save buys a lot of freedom — including the freedom to never read a "your usage exceeds the free tier" email again.